

Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 3	Tema: Fundamentos de Consumo de APIs con Axios y React
	Semana 12	Tutor: Ing. Victoria Castro

Introducción a Axios:

¿Qué es Axios?

Axios es una biblioteca de JavaScript que funciona como un cliente HTTP basado en Promesas. Se utiliza tanto en el navegador como en entornos de Node.js para realizar peticiones a servicios web o APIs (Application Programming Interfaces) de forma sencilla y eficiente.

Instalación de Axios

Para integrar Axios en un proyecto de React, se debe instalar a través del gestor de paquetes de Node (npm) ejecutando el siguiente comando en la terminal del proyecto:

npm install axios

Diferencia entre Fetch y Axios

Aunque el navegador web incluye de forma nativa la API `fetch()`, Axios ofrece varias ventajas que facilitan el desarrollo:

Característica	API Nativa <code>fetch()</code>	Biblioteca Axios
Transformación JSON	Manual (requiere hacer <code>.json()</code>).	Automática (los datos ya vienen listos en formato JSON).
Manejo de Errores	No rechaza la promesa en errores HTTP (ej. 404 o 500). Requiere evaluar <code>response.ok</code> .	Rechaza la promesa automáticamente si el estatus está fuera del rango 2xx (va directo al <code>catch</code>).
Soporte de Navegadores	Bueno, pero puede requerir <i>polyfills</i> en navegadores muy antiguos.	Excelente compatibilidad hacia atrás de forma nativa.
Cancelación de peticiones	Complejo (requiere <code>AbortController</code>).	Simple e integrado.

Ventajas principales de Axios

- **Interceptores:** Permite transformar la petición o la respuesta antes de que sean manejadas por el `then` o el `catch` (ideal para adjuntar tokens automáticamente).
- **Protección CSRF:** Trae seguridad integrada para proteger contra falsificación de peticiones en sitios cruzados.
- **Configuración global:** Permite definir una URL base (`baseUrl`) y cabeceras por defecto para no repetirlas en cada petición.

Sección I: Realizando una Petición GET

¿Qué es una petición GET?

Se utiliza para solicitar o recuperar datos de un servidor web sin modificar el estado del recurso.

Estructura básica con `axios.get()`

Cuando realizamos una petición con Axios, la respuesta del servidor siempre se envuelve en un objeto que contiene la propiedad `.data`, que es donde reside la información útil devuelta por la API.

```
import axios from 'axios';
```

```
axios.get('https://api.example.com/users')  
  .then((response) => {  
    // La conversión a JSON es automática, los datos están en response.data  
    console.log(response.data);  
  })  
  .catch((error) => {  
    console.error("Hubo un error en la petición:", error);  
  });
```

Sección 2: Manejo de Estados en React al Consumir APIs

Las peticiones HTTP son asíncronas y toman tiempo. Por lo tanto, una aplicación de React debe gestionar tres estados fundamentales para ofrecer una buena experiencia de usuario:

1. **Estado para almacenar datos (data):** Guarda la respuesta exitosa de la API.
2. **Estado de carga (loading):** Un valor booleano (true/false) que indica si la petición sigue en proceso para mostrar un spinner o texto de "Cargando...".
3. **Manejo de errores (error):** Almacena el mensaje o el objeto de error si algo sale mal (ej. problemas de red o servidor caído), permitiendo mostrar una alerta en pantalla.

Estructura de estados típica:

```
const [users, setUsers] = useState([]);  
const [loading, setLoading] = useState(true);  
const [error, setError] = useState(null);
```

Sección 3: Uso de useEffect para Consumir APIs

¿Qué hace useEffect?

useEffect es un Hook de React que permite ejecutar **efectos secundarios** en componentes funcionales. Actividades como llamadas a APIs, suscripciones o modificaciones manuales del DOM no pertenecen al flujo de renderizado directo y deben controlarse aquí.

Ejecutar peticiones al cargar el componente

Para que una API se consulte inmediatamente cuando el usuario entra a la pantalla, se debe estructurar el useEffect con un **array de dependencias vacío []**. Esto le indica a React que ejecute el código una única vez tras el primer renderizado.

Ejemplo Práctico Integrado (GET)

JavaScript

```
import React, { useState, useEffect } from 'react';  
import axios from 'axios';
```

```
function UserList() {  
  const [users, setUsers] = useState([]);  
  const [loading, setLoading] = useState(true);
```

```
const [error, setError] = useState(null);

useEffect(() => {
  // Definimos una función asíncrona dentro del efecto
  const fetchUsers = async () => {
    try {
      setLoading(true); // Iniciamos la carga
      const response = await axios.get('https://jsonplaceholder.typicode.com/users');
      setUsers(response.data); // Guardamos los datos
    } catch (err) {
      setError(err.message); // Guardamos el error si ocurre
    } finally {
      setLoading(false); // Finalizamos la carga pase lo que pase
    }
  };

  fetchUsers();
}, []); // Array vacío = Ejecuta solo al montar el componente

if (loading) return <p>Cargando usuarios...</p>;
if (error) return <p>Error al cargar datos: {error}</p>;

return (
  <ul>
    {users.map(user => (
      <li key={user.id}>{user.name}</li>
    ))}
  </ul>
);
}
```

Sección 4: Peticiones POST Básicas

¿Qué es una petición POST?

Se utiliza para enviar datos nuevos al servidor con el fin de crear un recurso (por ejemplo, registrar un nuevo usuario, enviar un formulario o publicar un artículo).

Enviar datos desde un formulario

Para enviar datos, capturamos la información de los inputs del formulario usando el estado de React (useState) y la pasamos como segundo argumento en el método axios.post().

Ejemplo Práctico (POST)

```
import React, { useState } from 'react';
import axios from 'axios';

function CreateUser() {
  // Capturar información con useState
  const [name, setName] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault(); // Evita que la página se recargue

    try {
      // Enviamos el objeto con los datos al backend
      const response = await axios.post('https://jsonplaceholder.typicode.com/users', {
        name: name
      });
      alert(`Usuario creado con éxito. ID: ${response.data.id}`);
      setName(""); // Limpiamos el input
    } catch (error) {
      console.error("Error al enviar datos:", error);
    }
  };
}
```

```
return (  
  <form onSubmit={handleSubmit}>  
    <label>Nombre del usuario:</label>  
    <input  
      type="text"  
      value={name}  
      onChange={(e) => setName(e.target.value)}  
    />  
    <button type="submit">Guardar Usuario</button>  
  </form>  
);  
}
```

Sección 5: Organización del Código y Buenas Prácticas

Separar componentes

A medida que la aplicación crece, mantener el diseño visual (UI) y las peticiones a la API en el mismo archivo se vuelve caótico. Es vital separar las responsabilidades:

- **Componentes de UI:** Se encargan exclusivamente de pintar los botones, tablas y formularios.
- **Servicios / API layers:** Archivos independientes (ej. `src/services/userService.js`) encargados únicamente de las llamadas a Axios.

```
// src/services/userService.js
```

```
import axios from 'axios';
```

```
const API_URL = 'https://jsonplaceholder.typicode.com';
```

```
export const getUsers = () => axios.get(`${API_URL}/users`);
```

```
export const createUser = (userData) => axios.post(`${API_URL}/users`, userData);
```

Buenas prácticas iniciales

- **Nombres claros de variables y funciones:** Usa nombres que describan la acción exacta. En lugar de `const [data, setData] = useState()`, prefiere nombres semánticos como `const [products, setProducts] = useState()`. Para las funciones, usa verbos de acción: `fetchProducts()`, `handleDeleteUser()`, `submitForm()`.
- **Manejar siempre el bloque catch:** Nunca asumas que la API funcionará al 100%. Un corte de internet o un servidor en mantenimiento romperá la aplicación si no controlas el error de Axios.
- **Limpieza de variables de entorno:** Almacena tus URLs base en archivos `.env` (ej. `REACT_APP_API_URL`) para no exponer las direcciones de tus servidores en el código fuente de forma estática.

Bibliografía y Fuentes Oficiales

- **Axios Official Documentation:** [Axios Docs](#)
 - *La guía y referencia oficial para conocer todos los métodos de configuración, interceptores y uso de Axios.*
- **React Official Documentation - Synchronizing with Effects:** [React.dev - useEffect](#)
 - *Documentación oficial de React que explica de manera detallada cómo y cuándo utilizar el Hook `useEffect` para efectos secundarios como llamadas a APIs.*
- **MDN Web Docs - Using Fetch:** [Mozilla Developer Network](#)
 - *Recurso base para comprender el estándar nativo de peticiones en la web y contrastarlo con clientes HTTP de terceros.*